

T.C.
MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ



Gömülü Linux Telemetry Sistemi ve Sürücü Analizi

SONUÇ RAPORU

Araştırmacıların Adı Soyadı
Birimi/Bölümü
Ferhat Kara 170425508
Bilgisayar MÜHENDİSLİĞİ

MAYIS 2026
İSTANBUL

İÇİNDEKİLER

1. GİRİŞ VE PROJE ÖZETİ

- 1.1. Projenin Amacı ve Kapsamı
- 1.2. Hedeflenen Sistem Mimarisi (ARMv7 / Cortex-A9)
- 1.3. Proje Klasör Hiyerarşisi ve Organizasyonu

2. AŞAMA 1: TELEMETRİ UYGULAMASI VE ÇAPRAZ DERLEME (CROSS-COMPILATION)

- 2.1. telemetry.c Kaynak Kod Tasarımı ve Modüler Yapı
- 2.2. POSIX Standartları ve Kütüphane Görünürlük Kısıtlamaları
- 2.3. strict C11 Derleme Hatalarının Analizi (Implicit Declaration & Sigaction)
- 2.4. arm-linux-gnueabi-hf-gcc ile Çapraz Derleme ve ELF Çıktı Doğrulaması

3. AŞAMA 2: KÖK DOSYA SİSTEMİNİN (ROOTFS) BUSYBOX İLE İNŞASI

- 3.1. Gömülü Sistemlerde RootFS ve Init Sürecinin Önemi
- 3.2. BusyBox Kaynak Kodunun Statik Olarak Yapılandırılması (CONFIG_STATIC=y)
- 3.3. make_rootfs.sh Otomasyon Betiğinin Çalışma Prensipleri
- 3.4. WSL ve Windows NTFS Dosya Sistemi Kısıtlamaları (mknod ve Karakter Boşluğu Krizleri)
- 3.5. ext4 Saf Linux Alanına Geçiş ve Ramdisk (rootfs.cpio.gz) Boyut Optimizasyonu

4. AŞAMA 3: EMÜLASYON VE DEPO UYUMSUZLUKLARI (QEMU DEBUGGING)

- 4.1. QEMU Emülasyon Parametrelerinin Teknik Analizi
- 4.2. Çevrimiçi Çekirdek (zImage) Tedarik Krizleri ve Broken Link Analizleri
- 4.3. Ubuntu 24.04 (Noble) Dağıtımının 32-bit ARM Depo Kısıtlaması (Unable to Locate Package)
- 4.4. Multiarch Altyapısının Kurulması ve ports.ubuntu.com Haritalandırması
- 4.5. Lokal Çekirdek Yapılandırması ve Başarılı Boot Süreci

5. AŞAMA 4: PROSESLER ARASI HABERLEŞME VE BUFFER OPTİMİZASYONU (IPC)

- 5.1. telemetry_ipc.c Süreç İskeleti (fork() ve Pipe() Mekanizması)
- 5.2. Sinyal Güvenliği (SIGINT/SIGTERM) ve Graceful Shutdown Lojisi
- 5.3. Linux Çekirdeği Tam Arabellek (Fully-Buffered) Tıkanıklık Bug'u
- 5.4. setvbuf() ve Line-Buffered Zorlaması ile Canlı Test ve Senkronizasyon

6. AŞAMA 5: CİHAZ AĞACI MANİPÜLASYONU (DEVICE TREE MANIPULATION)

- 6.1. Donanım Soyutlama Katmanı Olarak Device Tree Mimarisi
- 6.2. Custom GPIO LED Düğümü Sözdizimi Analizi
- 6.3. compatible = "gpio-leds" Özelliği ve Çekirdek Sürücü Eşleşme Teorisi
- 6.4. sysfs Arayüzü Üzerinden Donanım Kontrolü

7. AŞAMA 6: KARAKTER SÜRÜCÜ (CHARACTER DRIVER) TEKNİK ANALİZİ

- 7.1. Karakter Sürücü Yaşam Döngüsü (module_init ve module_exit)
- 7.2. Sanal Dosya Sistemi (VFS) ve file_operations Yapısı Haritalandırması
- 7.3. Kullanıcı Alanı ve Çekirdek Alanı Bellek İzolasyonu
- 7.4. copy_to_user() ve copy_from_user() Güvenlik Mekanizmaları

8. SONUÇ VE KAZANIMLAR

1. GİRİŞ VE PROJE ÖZETİ

1.1. Projenin Amacı ve Kapsamı

Bu projenin temel amacı; donanım kısıtlı gömülü sistemlerin mimari altyapısını, işletim sistemi katmanlarını ve çekirdek (kernel) programlama dinamiklerini pratik olarak simüle etmek, analiz etmek ve doğrulamaktır. Proje kapsamında, endüstriyel gömülü sistem senaryolarında sıkça karşılaşılan telemetri veri toplama süreçleri modellenmiştir. Bu bağlamda, heterojen veri tiplerinin (Sıcaklık, Sistem Yüku ve Durum Flag'i) güvenli bir şekilde üretilmesi, Süreçler Arası İletişim (IPC) kanalları vasıtasıyla asenkron ve kayıpsız olarak aktarılması hedeflenmiştir.

Projenin teknik kapsamı; hedef donanım mimarisi için çapraz derleme (cross-compilation) yapılması, BusyBox tabanlı minimal bir kök dosya sisteminin (RootFS) sıfırdan inşa edilmesi, adlandırılmış boru hatları (FIFO/Named Pipes) üzerinden Linux tampon bellek (buffer) yönetiminin optimize edilmesi, donanım soyutlama katmanı olan Cihaz Ağacı (Device Tree) yapılarının tasarlanması ve kullanıcı alanı-çekirdek alanı bellek izolasyonunu koruyan Karakter Sürücü (Character Driver) mimarilerinin teorik incelemelerini sentezlemektedir. Proje boyunca karşılaşılan işletim sistemi tıkanıklıkları (NTFS yetki kısıtları, multiarch bağımlılık çakışmaları ve çekirdek seviyesindeki tam arabellek blokajları) birer sistem mühendisliği problemi olarak ele alınmış, analiz edilmiş ve optimize çözümler üretilmiştir.

1.2. Hedeflenen Sistem Mimarisi (ARMv7 / Cortex-A9)

Gömülü Linux ekosisteminde, donanıma bağımlı yazılımların geliştirilmesi ve test edilmesi süreçlerinde hedef donanım mimarisinin seçimi ve simülasyon stratejisi kritik bir öneme sahiptir. Bu projede, endüstriyel otomasyon, otomotiv elektroniğı ve ağ altyapılarında yaygın bir olgunluğa sahip olan ARMv7-A (32-bit ARM Architecture) mimarisi ve bu mimarinin en kararlı çekirdek tasarımı olan Cortex-A9 işlemcisi temel alınmıştır.

Cortex-A9 çekirdeğı; donanımsal düzeyde çok çekirdek (Symmetric Multiprocessing - SMP) desteğı, gelişmiş boru hattı (pipeline) yapısı ve bellek yönetim birimi (MMU - Memory Management Unit) barındırmaktadır. Gömülü Linux çekirdeğinin sanal bellek adreslemesi yapabilmesi, süreçleri (process) birbirinden izole edebilmesi ve kullanıcı alanındaki (userspace) bir çökmenin çekirdek alanını (kernel space) etkilemesini engellemesi tamamen MMU donanımının varlığına bağılıdır. Projede emülasyon ortamı olarak seçilen QEMU (Quick Emulator) mimarisindeki VExpress-A9 (Versatile Express) geliştirme kartı şablonu, Cortex-A9 işlemcisini, GPIO denetleyicilerini, sistem zamanlayıcılarını ve seri konsol haberleşmesini (UART - Universal Asynchronous Receiver-Transmitter) tam uyumlu olarak taklit etmektedir. Bu sayede, fiziksel bir donanıma ihtiyaç duyulmaksızın, hedef mimari için üretilen binary dosyalarının komut setleri (instruction sets) doğrudan host makinede yorumlanabilmektedir.

1.3. Proje Klasör Hiyerarşisi ve Organizasyonu

Gömülü Linux projelerinde dosya sisteminin ve kaynak kodların organizasyonu, projenin taşınabilirliğı, bakım kolaylığı ve Linux standart hiyerarşisine (FHS - Filesystem Hierarchy Standard) uyumu açısından titizlikle yürütülmelidir. Proje süresince Windows Subsystem for Linux (WSL) üzerindeki NTFS disk bölümlerinin boşluk içeren dosya yolları (/mnt/c/users/ferhat kara/...) ve Linux özel dosya sistemi izinleri (mknode) üzerindeki kısıtlamaları analiz edilmiş ve proje klasör mimarisi tamamen saf Linux ext4 ev dizini (~/.works/170425508_ferhatkara/) altına taşınmıştır.

```
(venv-ardupilot) ferhatkara@FERHAT:~$ ls
4 ardupilot hello hello.c istikbal-ih sel.sh venv-ardupilot work.sh works
(venv-ardupilot) ferhatkara@FERHAT:~$ cd works
(venv-ardupilot) ferhatkara@FERHAT:~/works$ ls
first.c
(venv-ardupilot) ferhatkara@FERHAT:~/works$ 170425508_ferhatkara
170425508_ferhatkara: command not found
(venv-ardupilot) ferhatkara@FERHAT:~/works$ cd 170425508_ferhatkara
-bash: cd: 170425508_ferhatkara: No such file or directory
(venv-ardupilot) ferhatkara@FERHAT:~/works$ mkdir 170425508_ferhatkara
(venv-ardupilot) ferhatkara@FERHAT:~/works$ cd 170425508_ferhatkara
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ ls
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ mkdir -p embedded_project/{source_code,rootfs,qemu}
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ ls
embedded_project
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ |
```

Yukarıdaki ekran görüntüsünde doğrulanmış olan ve projenin hocaya teslim edilmesinde kılavuzluk edecek olan nihai klasör organizasyonu, işlevsel modülleriyle birlikte aşağıda teknik detaylarıyla şematize edilmiştir:

README.md: Sistem mimarisini, bağımlılıkları, çapraz derleme adımlarını ve iki farklı terminal penceresi üzerinden yürütülecek canlı IPC haberleşme test prosedürünü içeren resmi kullanıcı dökümantasyonudur.

prompts.txt: Geliştirme süreci boyunca karşılaşılan mimari hataları, paket yöneticisi kilitlenmelerini, arabellek (buffer) kaymalarını ve bunlara karşı geliştirilen debugging promptlarını kronolojik olarak içeren yapay zeka mühendislik günlüğüdür.

report.pdf: Projenin tüm pratik çıktılarının, teorik arka plan analizlerinin ve hata ayıklama dökümlerinin toplandığı bu akademik mühendislik raporudur.

screenshots/: Süreç boyunca karşılaşılan implicit declaration, Makefile durdurma hataları, 404 depo krizleri, tampon bellek blokajı sorunları ve başarılı canlı test çıktılarını içeren teknik görsel arşividir.

source_code/: Çapraz derleme süreçlerinin yürütüldüğü ana kod dizinidir. İçerisinde telemetry.c (verici/sensör simülasyonu) ve telemetry_ipc.c (alıcı/veri işleyici) kaynak kodları ile yerel mimari (local) ve hedef donanım mimarisi (ARMv7) için üretilmiş çalıştırılabilir (executable) binary dosyaları yer alır.

rootfs/: BusyBox tabanlı minimal kök dosya sisteminin el ile inşa edildiği alandır. İçerisinde tüm süreci otomatize eden ve bağımlılıkları temizleyen make_rootfs.sh Bash betiği barındırır.

dts/: Gömülü donanım soyutlama katmanı analizini içeren klasördür. İçerisindeki telemetry_led.dts dosyası, durum LED'ini belirli bir GPIO pinine haritalayan cihaz ağacı kaynak kodunu içerir.

qemu/: Simülasyonun yürütüldüğü katmandır. Paketlenen minimal kök dosya sistemi imajı (rootfs.cpio.gz) donanımı boot etmek üzere bu klasörde muhafaza edilir.

2. AŞAMA 1: TELEMETRİ UYGULAMASI VE ÇAPRAZ DERLEME (CROSS-COMPILATION)

2.1. telemetry.c Kaynak Kod Tasarımı ve Modüler Yapı

Gömülü Linux platformlarında çalışan yazılımların, sistem kaynaklarını (CPU ve RAM) minimum düzeyde tüketmesi ve çalışma anında deterministik bir kararlılık sergilemesi zorunludur. Bu doğrultuda tasarlanan telemetry.c uygulaması, monolitik bir yapı yerine Unix felsefesine uygun olarak modüler, tahmini zor olmayan ve olay tabanlı (event-driven) bir mimariyle kurgulanmıştır.

Uygulamanın ana görevi; endüstriyel bir uç cihazın (edge device) durumunu temsil eden heterojen ham verileri simüle etmek ve bu verileri saniyede bir standart çıktı (stdout) akışına yapılandırılmış bir string formatında basmaktır. Sistemde modellenen veri yapısı (data structure) aşağıdaki gibidir:

- **Sıcaklık (Temperature - float):** Ondalıkli sayı hassasiyetinde, uç cihazın sensör verisini temsil eder (\$20.0^{\circ}\text{C}\$ ile \$85.0^{\circ}\text{C}\$ arasında dalgalanır).
- **Sistem Yüğü (System Load - int):** \$0\$ ile \$100\$ arasında tam sayı değeri alan, işlemci yüğü simülasyonudur.
- **Sistem Durumu (Status Flag - bool):** Sıcaklık ve yüğü değerlerinin kritik eşikleri (Sıcaklık \$75.0^{\circ}\text{C}\$ ve Yüğü \$90\$) aşp aşmadığını denetleyen mantıksal bayraktır.

Kod mimarisinde bellek sızıntılarını (memory leaks) engellemek amacıyla dinamik bellek yönetimi (malloc/free) yerine tamamen yığın (stack) bellek adreslemesi tercih edilmiştir. Zaman damgası üretimi için thread-safe ve reentrant özelliklere sahip olan localtime_r fonksiyonu kullanılarak çoklu görev ortamlarında (multitasking) yarış durumları (race conditions) engellenmiştir.

2.2. POSIX Standartları ve Kütüphane Görünürlük Kısıtlamaları

Yazılımın taşınabilirliğini (portability) garanti altına almak adına, endüstriyel işletim sistemi standardı olan **POSIX (Portable Operating System Interface)** kurallarına tam uyum sağlanmıştır. POSIX standartları, yazılımların farklı Unix/Linux türevlerinde kaynak kod seviyesinde uyumlu çalışmasını hedefler.

Ancak GNU C Kütüphanesi (glibc), standart C başlık dosyalarını (<stdio.h>, <time.h>, <signal.h>) ayrıştırırken derleyicinin parametrelerine göre belirli fonksiyonların görünürlüğünü maskeler ya da açar. Kod geliştirilirken standart C11 kuralları kesin olarak zorlandığında (-std=c11), glibc kütüphanesi POSIX tabanlı sistem çağrılarını ve uzantılarını dışarıya kapatır. Bu kısıtlamayı aşmak ve işletim sisteminin alt çekirdek API'lerini derleyiciye görünür kılmak için **Özellik Test Makroları (Feature Test Macros)** kullanılmalıdır. Kodun en üstüne, hiçbir kütüphane dahil edilmeden önce yerleştirilen #define _POSIX_C_SOURCE 200809L makrosu, derleyiciye işletim sisteminin POSIX.1-2008 standardındaki tüm gelişmiş süreç kontrol ve sinyal yönetimi kütüphane prototiplerini aktif etme talimatı verir.

2.3. strict C11 Derleme Hatalarının Analizi (Implicit Declaration & Sigaction)

Projenin ilk geliştirme safhasında, özellik test makroları dahil edilmeden katı C11 standardı zorlanarak yapılan çapraz derleme girişimi, GNU derleyicisinin katı kuralları neticesinde başarısızlıkla sonuçlanmıştır.

```
ferhatkara@FERHAT: /mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara$ ls
README.md  prompts.txt  README  README  README
ferhatkara@FERHAT: /mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara$ cd source_code
ferhatkara@FERHAT: /mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/source_code$ ls
telemetry.c
ferhatkara@FERHAT: /mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/source_code$ arm-linux-gnueabihf
-gcc -Wall -Wextra -O2 -std=c11 telemetry.c -o telemetry
telemetry.c: In function 'get_timestamp':
telemetry.c:97:5: warning: implicit declaration of function 'localtime_r'; did you mean 'localtime'? [-Wimplicit-function-declaration]
   97 |     localtime_r(&now, &tm_info);
      |     ^
      |     localtime
telemetry.c: In function 'telemetry_init':
telemetry.c:166:22: error: storage size of 'sa' isn't known
   166 |     struct sigaction sa;
      |
telemetry.c:173:9: warning: implicit declaration of function 'sigaction' [-Wimplicit-function-declaration]
   173 |     if (sigaction(SIGINT, &sa, NULL) < 0)
      |
telemetry.c:166:22: warning: unused variable 'sa' [-Wunused-variable]
   166 |     struct sigaction sa;
      |
```

Yukarıdaki terminal çıktısında detaylı olarak görülen hataların kök neden analizi şu şekildedir:

1. **warning: implicit declaration of function 'localtime_r':** Derleyici, <time.h> kütüphanesini okumasına rağmen reentrant zaman fonksiyonu olan localtime_r prototipini bulamamıştır. C standardında bildirilmeyen fonksiyonlar örtülü (implicit) olarak kabul edilir ve gömülü sistem derleyicileri bu durumu bir risk olarak görerek uyarı fırlatır.
2. **error: storage size of 'sa' isn't known:** Gelişmiş sinyal yönetim yapısı olan struct sigaction sa; tanımı derleyici tarafından çözümlenememiştir. Derleyici, yapının (struct) bellekte ne kadar yer kaplayacağını (storage size) bilemediği için derleme işlemini doğrudan durdurmuştur (fatal error).
3. **warning: implicit declaration of function 'sigaction':** Güvenli sinyal yakalama arayüzü olan sigaction sistem çağrısı maskelendiği için derleyici fonksiyonun geri dönüş tipini ve argüman yapısını doğrulayamamıştır.

Bu üç kritik hata, yazılımın donanıma bağımlı bir mimari uyumsuzluğundan değil, tamamen glibc'nin katı standart maskeleye lojiğinden kaynaklanmıştır. Kodun başına eklenen `_POSIX_C_SOURCE` makrosu ve sinyal maskeleye kütüphanelerinin nizami dahil edilmesiyle kod düzeyindeki tüm bug'lar temizlenmiştir.

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdint.h>
#include <signal.h>
#include <time.h>
#include <unistd.h>
#include <string.h>

#define TELEMETRY_INTERVAL_SEC 1
#define TEMP_MIN_C 20.0f
#define TEMP_MAX_C 85.0f
#define LOAD_MIN 0
#define LOAD_MAX 100

typedef struct {
    float temperature_c;
    int system_load;
    bool status_ok;
} telemetry_data_t;

static volatile sig_atomic_t g_running = 1;

EOF return EXIT_SUCCESS; shutdown complete.\n"); ironment)\n\n"); &sa, NULL) < 0) {"OK" : "FAIL"};
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ ls
telemetry.c
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ |

return min + (rand() % (max - min + 1));
```

2.4. arm-linux-gnueabi-hf-gcc ile Çapraz Derleme ve ELF Çıktı Doğrulaması

Gömülü sistem mühendisliğinde, yazılımın derlendiği platform (Host Machine - x86_64) ile çalışacağı platform (Target Machine - ARMv7) farklıdır. Bu uyumsuzluğu aşmak amacıyla **Çapraz Derleyici (Cross-Compiler)** araç zincirleri kullanılır. Projede kullanılan arm-linux-gnueabi-hf-gcc derleyicisinin isimlendirme standartları şu mimari detayları ifade eder:

- arm: Hedef işlemci mimarisinin ARM olduğunu,
- linux: Hedef işletim sisteminin Linux çekirdeği olduğunu,
- gnueabi-hf: GNU C kütüphanesini kullandığını ve donanımsal yüzer nokta hesaplama birimine (Hard Float - VFP) sahip olduğunu belirtir.

Temizlenen kaynak kod, optimizasyon seviyesi seviye 2 (-O2), tüm uyarıları aktif etme (-Wall -Wextra) ve strict C11 (-std=c11) parametreleriyle çapraz derleme işlemine tabi tutulmuştur. Derleme sonucunda terminal herhangi bir warning veya error fırlatmadan pürüzsüzce alt satıra geçmiştir.

```
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ # ARMv7 cross-compiler ile derle
arm-linux-gnueabi-hf-gcc -Wall -Wextra -O2 -std=c11 telemetry.c -o telemetry

ELF dosya yapısını doğrula
ile telemetry
telemetry: ELF 32-bit LSB pie executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-arm
F.so.3, BuildID[sha1]=eb2394a75bec19605eefdb8ca2e2d5801a6da17b, for GNU/Linux 3.2.0, not stripped
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ |
```

Üretilen telemetry ikili (binary) dosyasının x86_64 host işlemcisinde çalıştırılmaya zorlanması durumunda işletim sistemi "Exec format error" hatası fırlatacaktır. Dosyanın hedef donanıma uygun üretildiğini mühendislik düzeyinde doğrulamak amacıyla Linux file komutu koşurulmuştur.

```
telemetry: cannot open 'telemetry' (No such file or directory)
ferhatkara@FERHAT:/mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/source_code$ file telemetry
telemetry: ELF 32-bit LSB pie executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-arm
hf.so.3, BuildID[sha1]=eb2394a75bec19605eefdb8ca2e2d5801a6da17b, for GNU/Linux 3.2.0, not stripped
ferhatkara@FERHAT:/mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/source_code$ |
```

Terminalden alınan file telemetry çıktısının analizi, çapraz derlemenin başarısını kesin olarak kanıtlamaktadır:

- **ELF 32-bit LSB pie executable:** Dosyanın Linux çalıştırılabilir formatı olan ELF (Executable and Linkable Format) standartlarında, 32-bit genişliğinde ve adres alanı yerleşim rastgeleleştirmesine uygun (PIE) üretildiğini,
- **ARM, EABI5 version 1 (SYSV):** Komut setinin doğrudan ARM işlemci mimarisine ve gömülü uygulama ikili arayüzü (EABI5) standartlarına göre mühürlendiğini,

- **dynamically linked, interpreter /lib/ld-linux-armhf.so.3:** Çalışma zamanında hedef gömülü Linux sistemindeki dinamik bağlayıcıya (ld-linux-armhf.so.3) ihtiyaç duyacağını doğrulamaktadır.

3. AŞAMA 2: KÖK DOSYA SİSTEMİNİN (ROOTFS) BUSYBOX İLE İNŞASI

3.1. Gömülü Sistemlerde RootFS ve Init Sürecinin Önemi

Bir Linux sisteminin başarıyla boot edebilmesi için yalnızca çekirdeğin (kernel) yüklenmesi yeterli değildir. Çekirdek, donanımı ilklendirdikten (initialization) hemen sonra kullanıcı alanındaki (userspace) ilk süreci başlatmak ve sistem hiyerarşisine erişmek zorundadır. İşte bu noktada **Kök Dosya Sistemi (RootFS)** devreye girer. RootFS; Linux çekirdeğinin mount ettiği, içinde temel sistem kütüphanelerini, yapılandırma dosyalarını, kabuk (shell) ortamını ve çalıştırılabilir binary dosyalarını barındıran en üst düzey dizin hiyerarşisidir.

Gömülü sistem açılış mimarisinde çekirdek, donanım sürücülerini ayağa kaldırdıktan sonra kök dosya sistemi içindeki ilk kullanıcı süreci olan **init** programını (/sbin/init veya /bin/sh) tetikler. init süreci, Linux işletim sisteminde **PID 1 (Process ID 1)** değerini alan, tüm diğer alt süreçlerin (child processes) atası olan en kritik mekanizmadır. Eğer çekirdek RootFS yapısına erişemezse veya PID 1 sürecini başlatamazsa, sistem **"Kernel Panic - not syncing: Attempted to kill init!"** hatası vererek tamamen kilitlenir. Gömülü Linux mimarilerinde RootFS'in minimal, hafif ve donanım kısıtlarına uygun tasarlanması, açılış hızını (boot time) ve kararlılığı doğrudan etkiler.

3.2. BusyBox Kaynak Kodunun Statik Olarak Yapılandırılması (CONFIG_STATIC=y)

Masaüstü veya sunucu Linux dağıtımlarında (Ubuntu, Debian vb.) RootFS yüzlerce megabayt yer kaplarken, gömülü sistemlerde bu lüks yoktur. Bu kısıtlamayı aşmak için gömülü sistem dünyasının İsviçre çakısı olarak bilinen **BusyBox** yazılımı kullanılır. BusyBox; standart bir Linux sisteminde bulunan yüzlerce temel Unix komutunu (ls, cd, mkdir, cp, init, sh vb.) tek bir küçük çalıştırılabilir ikili (single binary) dosya içinde birleştiren optimize bir projedir.

BusyBox kaynak kodunun yapılandırılması aşamasında make defconfig komutu ile varsayılan şablon üretilir. Ancak gömülü sistem inşasında en kritik mühendislik kararı, bu yapının **Statik (Static Binary)** olarak derlenmesidir. BusyBox konfigürasyon arayüzünde **CONFIG_STATIC=y** parametresi aktif edildiğinde, derleyici (gcc) komutların ihtiyaç duyduğu tüm C standart kütüphane fonksiyonlarını (glibc veya musl) üretilen ikili dosyanın içerisine fiziken gömer (static linking).

Eğer statik derleme yapılmazsa (dynamic linking), RootFS içerisine hedef mimariye uygun dinamik bağlayıcıların (ld-linux-armhf.so) ve paylaşımlı kütüphanelerin (libc.so) el ile taşınması gerekir. En ufak bir kütüphane sürüm uyumsuzluğunda sistem "missing shared libraries" hatasıyla çökecektir. CONFIG_STATIC=y seçimi, dinamik kütüphane bağımlılık riskini tamamen sıfırlayarak tek bir bağımsız binary ile sistemin ayağa kalkmasını garanti altına alır.

3.3. make_rootfs.sh Otomasyon Betiğinin Çalışma Prensipleri

Kök dosya sisteminin el ile oluşturulması süreçleri insan hatasına çok açıktır. Bu nedenle, tüm süreci otomatize eden kararlı bir Bash betiği (make_rootfs.sh) kurgulanmıştır. Betiğin algoritması saniyeler içinde şu kritik üretim adımlarını yürütür:

1. **Temizlik ve İndirme:** Varsa eski build kalıntılarını temizler, resmi Busybox sunucusundan kararlı kaynak kod paketini (busybox-1.36.1.tar.bz2) wget ile çeker ve dışarı çıkartır.
2. **Statik Konfigürasyon ve Derleme:** BusyBox kaynak kod dizinine girerek make defconfig üretir, .config dosyasına CONFIG_STATIC=y parametresini enjekte eder ve make -j\$(nproc) ile çapraz derlemeyi başlatır.
3. **Staging Alanına Kurulum:** Derlenen komut yapısını make install CONFIG_PREFIX=../rootfs komutuyla hedef geçici klasöre kurar.
4. **Hiyerarşi ve Script Enjeksiyonu:** Linux standartlarına (FHS) uygun olarak /dev, /etc, /proc, /sys dizinlerini açar. Sistem başlangıcında donanımları mount edecek olan /etc/init.d/rcS betiğini ve konsol lojigini yöneten /etc/inittab dosyasını oluşturup gerekli çalıştırma izinlerini (chmod +x) atar.
5. **cpio İmaj Mühürlemesi:** Tüm yapıyı cpio ve gzip algoritmalarından geçirerek QEMU'nun okuyabileceği rootfs.cpio.gz ramdisk paketine dönüştürür.

Betiğin ilk test aşamasında, çok satırlı blok genişletmesi (brace expansion) esnasında Bash yorumlayıcısının syntax hatasına düştüğü saptanmıştır (line 152: syntax error near unexpected token '}').

```
ferhatkara@FERHAT:/mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/rootfs$ chmod +x make_rootfs.sh
ferhatkara@FERHAT:/mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/rootfs$ ./make_rootfs.sh
./make_rootfs.sh: line 152: syntax error near unexpected token `}'
ferhatkara@FERHAT:/mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/rootfs$ |
```

Bu sözdizimi hatası, parantez içindeki boşlukların ve yeni satır karakterlerinin Bash tarafından yanlış yorumlanmasından kaynaklanmıştır. Komut satırları tekil, sıralı ve net yol tanımlarıyla (explicit paths) revize edilerek otomasyon scripti tamamen pürüzsüz çalışır hale getirilmiştir.

3.4. WSL ve Windows NTFS Dosya Sistemi Kısıtlamaları (mknod ve Karakter Boşluğu Krizleri)

Projenin en öğretici sistem mühendisliği krizi, Windows altındaki Linux katmanı (WSL) üzerinde, Windows dosya sistemi (/mnt/c/...) sınırlarında derleme yapmaya çalışırken yaşanmıştır. Bu süreçte iki temel kırılma noktası analiz edilmiştir:

1. **GNU Make ve Karakter Boşluğu Engeli:** Windows disk yollarında kullanıcı profilinden kaynaklı olarak boşluk karakteri yer almaktadır (/mnt/c/users/ferhat kara/...). BusyBox derleme motoru olan GNU Make, Makefile dosyalarını parse ederken yollardaki boşluk karakterlerini yeni bir argüman veya geçersiz hedef (invalid target pattern) olarak algılar ve sistemi kilitler.

```
ferhatkara@FERHAT:/mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/rootfs$ ./make_rootfs.sh
[INFO] Starting BusyBox RootFS build...
[INFO] Cleaning previous build...
[INFO] Downloading BusyBox 1.36.1...
--2026-05-18 16:54:52-- https://busybox.net/downloads/busybox-1.36.1.tar.bz2
Resolving busybox.net (busybox.net)... 140.211.167.122
Connecting to busybox.net (busybox.net)|140.211.167.122|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2525473 (2.4M) [application/x-bzip2]
Saving to: 'busybox-1.36.1.tar.bz2'

busybox-1.36.1.tar.bz2      100%[=====] 2.41M  1.23MB/s   in 2.0s

2026-05-18 16:54:55 (1.23 MB/s) - 'busybox-1.36.1.tar.bz2' saved [2525473/2525473]

[INFO] Extracting BusyBox...
[INFO] Generating BusyBox default config...
/mnt/c/users/ferhat:1: *** target pattern contains no '%'. Stop.
ferhatkara@FERHAT:/mnt/c/users/ferhat kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/rootfs$ |
```

2. **NTFS ve mknod İzin Krizleri:** Linux işletim sisteminde her şey bir dosyadır. Standart bir RootFS içinde terminal girdilerini ve çıktılarını simüle etmek için /dev/null (karakter cihazı, major 1, minor 3) ve /dev/console (major 5, minor 1) özel aygıt düğümlerinin (device nodes) mknod komutuyla fiziken oluşturulması gerekir. Ancak Windows'un temel dosya sistemi olan NTFS, Linux'un bu özel blok ve karakter aygıt düğümü özniteliklerini donanımsal düzeyde desteklemez. Bu yüzden sudo mknod komutları koşurulduğunda NTFS alanında "Operation not permitted" veya "cannot stat" izin hataları fırlatılır (Görsel 19 ve 20).

3.5. ext4 Saf Linux Alanına Geçiş ve Ramdisk (rootfs.cpio.gz) Boyut Optimizasyonu

Windows NTFS katmanının getirdiği bu aşamaz mimari engeller neticesinde, projenin tamamen sanal diskin içindeki saf Linux alanı olan **ext4** dosya sistemine (WSL'in kendi home dizini: ~/works/170425508_ferhatkara/) taşınmasına karar verilmiştir.

```
(venv-ardupilot) ferhatkara@FERHAT:~$ ls
4 ardupilot hello hello.c istikbal-ih sel.sh venv-ardupilot work.sh works
(venv-ardupilot) ferhatkara@FERHAT:~$ cd works
(venv-ardupilot) ferhatkara@FERHAT:~/works$ ls
first.c
(venv-ardupilot) ferhatkara@FERHAT:~/works$ 170425508_ferhatkara
170425508_ferhatkara: command not found
(venv-ardupilot) ferhatkara@FERHAT:~/works$ cd 170425508_ferhatkara
-bash: cd: 170425508_ferhatkara: No such file or directory
(venv-ardupilot) ferhatkara@FERHAT:~/works$ mkdir 170425508_ferhatkara
(venv-ardupilot) ferhatkara@FERHAT:~/works$ cd 170425508_ferhatkara
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ ls
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ mkdir -p embedded_project/{source_code,rootfs,qemu}
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ ls
embedded_project
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara$ |
```

Proje ext4 alanına taşındığı anda, dosya yollarındaki boşluk krizleri ortadan kalkmış, GNU Make motoru pürüzsüzce çalışmış ve mknod komutları Linux çekirdeğinin sanal dosya sistemi standartlarına uygun olarak başarıyla aygıt düğümlerini oluşturmuştur.

Klasör hiyerarşisi tam anlamıyla güvenceye alındıktan sonra, BusyBox'ın kapladığı alan du -s komutuyla analiz edilmiştir. İmaj boyutu el yordamıyla büyük seçilerek donanım şişirilmemiş, tam

kapladığı hacim hesaplanarak minimal düzeyde esneklik payıyla **23-24 blok** arasında nokta atışı bir boyutla paketlenmiştir.

[GÖRSEL YERLEŞTİRME: Buraya screen_shoots klasörü içerisinde 21_stage2_rootfs_23blocks_success.png (eski adıyla 21.png) görselini yerleştiriniz.]

Yukarıdaki ekran çıktısında belgelendiği üzere, RootFS hiyerarşisi saf Linux ext4 alanında 23 blocks şeklinde optimize bir boyutla mühürlenerek QEMU emülasyonuna hazır, kusursuz bir initial ramdisk (initrd) haline getirilmiştir.

4. AŞAMA 3: EMÜLASYON VE DEPO UYUMSUZLUKLARI (QEMU DEBUGGING)

4.1. QEMU Emülasyon Parametrelerinin Teknik Analizi

Gömülü Linux geliştirme süreçlerinde, hedef donanım (Target hardware) üzerinde test yapmadan önce sanallaştırma katmanında doğrulama yapmak, geliştirme maliyetlerini düşürür ve donanımsal arıza risklerini sıfırlar. Bu projede, ARMv7 mimarisini simüle etmek amacıyla açık kaynaklı donanım emülatörü olan QEMU (Quick Emulator) kullanılmıştır.

Sistemi ayağa kaldırmak için kurgulanan ham qemu-system-arm komut seti ve bu komut setinde yer alan bayrakların (flags) mikro mimari düzeyindeki teknik analizleri şu şekildedir:

-M vexpress-a9: QEMU'ya simüle edilecek anakart şablonunun ARM Versatile Express, üzerindeki işlemcinin ise Cortex-A9 çekirdeği olduğunu bildirir. Bu şablon; sistem zamanlayıcılarını, kesme denetleyicisini (GIC) ve UART seri haberleşme haritalamasını donanım seviyesinde taklit eder.

-cpu cortex-a9: İşlemci çekirdeğinin komut setini (Instruction set) tam olarak Cortex-A9 moduna zorlar.

-m 256M: Sanal makineye 256 Megabayt fiziksel RAM tahsis eder. Gömülü Linux mimarisi için bu hacim, kernel ve hafif RootFS katmanlarının RAM disk üzerinde şişmeden çalışması için fazlasıyla optimize bir alandır.

-kernel zImage: Çekirdeğin sıkıştırılmış binary imajını (zImage) doğrudan QEMU'nun sanal bellek adres alanına yükler ve bootloader simülasyonunu atlayarak doğrudan kernel giriş noktasını (entry point) tetikler.

-dtb vexpress-v2p-ca9.dtb: Donanım bileşenlerinin adres haritalarını ve kesme hatlarını barındıran derlenmiş Cihaz Ağacı (Device Tree Blob) dosyasını çekirdeğe parametre olarak mühürlar.

-initrd rootfs.cpio.gz: El ile inşa ettiğimiz sıkıştırılmış minimal kök dosya sistemini, geçici RAM disk (Initial Ramdisk) olarak donanıma bağlar.

-append "console=ttyAMA0 rdinit=/sbin/init": Çekirdeğe geçirilecek komut satırı argümanlarıdır (Kernel cmdline). console=ttyAMA0, Linux'un tüm sistem çıktılarını ARM Versatile Express üzerindeki ilk donanımsal seri porta (UART) yönlendirmesini söyler. rdinit=/sbin/init, çekirdeğin RAM diski mount ettikten sonra tetikleyeceği ilk süreç (PID 1) yolunu kesin olarak dikte eder.

-nographic: Grafiksel arayüz (VGA) simülasyonunu kapatarak sanal makinenin tüm giriş ve çıkışlarını doğrudan mevcut terminale (stdin/stdout) bağlar.

4.2. Çevrimiçi Çekirdek (zImage) Tedarik Krizleri ve Broken Link Analizleri

Projenin emülasyon safhasında karşılaşılan en büyük pratik tıkanıklık, sanal donanımı boot edecek olan uygun bir Linux çekirdeğinin (zImage) ve cihaz ağacının (.dtb) tedarik edilmesi esnasında yaşanmıştır. İlk denemelerde, internet üzerindeki açık kaynaklı test repolarından (raw.githubusercontent.com/... veya boot.shifter.media/...) wget vasıtasıyla hazır imajlar çekilmeye çalışılmıştır.

```
ferhatkara@FERHAT: /mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/ferhat_kara_embedded_project$ #  
QEMU klasörümüze gidelim  
cd ~/ferhat_kara_embedded_project/qemu  
  
# Ubuntu/Debian depolarından vexpress için hazır zImage ve DTB içeren paketi indirelim  
wget https://raw.githubusercontent.com/wookeeyorg/ers/master/zImage  
wget https://raw.githubusercontent.com/wookeeyorg/ers/master/vexpress-v2p-ca9.dtb  
  
# RootFS imajımızı da kolaylık olsun diye bu klasöre kopyalayalım  
cp ~/ferhat_kara_embedded_project/rootfs/rootfs.cpio.gz .  
--bash: cd: /home/ferhatkara/ferhat_kara_embedded_project/qemu: No such file or directory  
--2026-05-18 17:11:59-- https://raw.githubusercontent.com/wookeeyorg/ers/master/zImage  
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.109.133, 185.199.110.133, ..  
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)[185.199.111.133]:443... connected.  
HTTP request sent, awaiting response... 404 Not Found  
2026-05-18 17:12:00 ERROR 404: Not Found.  
  
--2026-05-18 17:12:00-- https://raw.githubusercontent.com/wookeeyorg/ers/master/vexpress-v2p-ca9.dtb  
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.110.133, 185.199.109.133, ..  
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)[185.199.108.133]:443... connected.  
HTTP request sent, awaiting response... 404 Not Found  
2026-05-18 17:12:00 ERROR 404: Not Found.  
  
cp: cannot stat '/home/ferhatkara/ferhat_kara_embedded_project/rootfs/rootfs.cpio.gz': No such file or directory  
ferhatkara@FERHAT: /mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/ferhat_kara_embedded_project$ |
```

Ancak sunuculardaki SSL sertifika uyumsuzlukları (Görsel 9) ve internet üzerindeki sihirli linklerin zamanla geçerliliğini yitirmesi neticesinde wget istekleri sürekli olarak HTTP request sent, awaiting response... 404 Not Found hatasıyla sonuçlanmıştır.

```
ferhatkara@FERHAT: /mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/ferhat_kara_embedded_project/qem  
u$ # Şu an bulunduğum qemu klasöründe kal, şu komutları ateşle:  
wget --no-check-certificate https://boot.shifter.media/current/armhf/vexpress/zImage  
wget --no-check-certificate https://boot.shifter.media/current/armhf/vexpress/vexpress-v2p-ca9.dtb  
--2026-05-18 17:52:33-- https://boot.shifter.media/current/armhf/vexpress/zImage  
Resolving boot.shifter.media (boot.shifter.media)... 192.0.78.201, 192.0.78.144  
Connecting to boot.shifter.media (boot.shifter.media)[192.0.78.201]:443... connected.  
WARNING: no certificate subject alternative name matches  
requested host name 'boot.shifter.media'.  
HTTP request sent, awaiting response... 404 Not Found  
2026-05-18 17:52:33 ERROR 404: Not Found.  
  
--2026-05-18 17:52:33-- https://boot.shifter.media/current/armhf/vexpress/vexpress-v2p-ca9.dtb  
Resolving boot.shifter.media (boot.shifter.media)... 192.0.78.144, 192.0.78.201  
Connecting to boot.shifter.media (boot.shifter.media)[192.0.78.144]:443... connected.  
WARNING: no certificate subject alternative name matches  
requested host name 'boot.shifter.media'.  
HTTP request sent, awaiting response... 404 Not Found  
2026-05-18 17:52:33 ERROR 404: Not Found.  
  
ferhatkara@FERHAT: /mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/ferhat_kara_embedded_project/qem  
u$ cp ../rootfs/rootfs.cpio.gz .  
cp: cannot stat '../rootfs/rootfs.cpio.gz': No such file or directory  
ferhatkara@FERHAT: /mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/ferhat_kara_embedded_project/qem  
u$ ls -lh  
total 0  
ferhatkara@FERHAT: /mnt/c/users/ferhat_kara/onedrive/masaüstü/cccc/_170425508_ferhatkara/ferhat_kara_embedded_project/qem  
u$ |
```

İnternet üzerindeki harici dosya bağlantılarına bağımlı kalmanın (broken link riski) sürdürülebilir bir sistem mühendisliği yaklaşımı olmadığı bu krizle kanıtlanmıştır. Dış dünyaya olan bağımlılığı sıfırlamak adına, hedef mimariye uygun kernel ve busybox paketlerinin tamamen yerel paket yöneticisi (apt) katmanı üzerinden, resmi işletim sistemi depoları vasıtasıyla çekilmesi kararlaştırılmıştır.

4.3. Ubuntu 24.04 (Noble) Dağıtımının 32-bit ARM Depo Kısıtlaması (Unable to Locate Package)

Yerel paket yöneticisine geçilip apt download linux-image-generic:armhf veya apt download busybox-static:armhf komutları çalıştırıldığında sistem bu kez çok daha derin bir mimari engelle karşı karşıya kalmıştır. Paket yöneticisi paketleri bulamamış ve E: Unable to locate package linux-image-generic:armhf hatasını fırlatmıştır.

```
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/qemu$ # Şu an bulunduğun qemu klasöründe olduğundan emin ol
cd ~/works/170425508_ferhatkara/embedded_project/qemu

# Ubuntu'nun resmi ARMhf kernel paketini yerel klasöre indirelim (Sisteme kurmaz, sadece indirir)
apt download linux-image-generic:armhf

# İndirilen paketin içindeki zImage ve cihaz ağacını (DTB) buraya çıkartalım
ar x linux-image-*.deb
tar -xzf data.tar.xz

# Dosyaları qemu klasörümüzün kök dizinine kopyalayıp isimlerini düzeltelim
cp boot/vmlinuz-* zImage
cp usr/lib/linux-image-*/vexpress-v2p-ca9.dtb .

# Ortalık temizlensin diye gereksiz ayıklama klasörlerini silelim
rm -rf boot usr lib data.tar.xz control.tar.gz debian-binary linux-image-*.deb
E: Unable to locate package linux-image-generic:armhf
ar: linux-image-*.deb: No such file or directory
tar: data.tar.xz: Cannot open: No such file or directory
tar: Error is not recoverable: exiting now
cp: cannot stat 'boot/vmlinuz-*': No such file or directory
cp: cannot stat 'usr/lib/linux-image-*/vexpress-v2p-ca9.dtb': No such file or directory
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/qemu$
```

Bu krizin arkasındaki temel işletim sistemi lojiği şu şekildedir: Host makinede çalışan WSL Ubuntu dağıtımı varsayılan olarak x86_64 (AMD64) mimarisine sahiptir ve apt mekanizması yalnızca bu mimarinin indeks listelerini tarar. Ayrıca, Ubuntu 24.04 (Noble Numbat) dağıtımı ile birlikte Canonical, 32-bit ARM (armhf) mimarisine olan desteğini resmi ana sunucu depolarından (archive.ubuntu.com) tamamen çekmiştir. Bu nedenle standart bir apt update çekildiğinde armhf dizinleri 404 hatası vererek sistemi kilitlemektedir.

```
Ign:10 http://archive.ubuntu.com/ubuntu noble/universe armhf Packages
Ign:11 http://archive.ubuntu.com/ubuntu noble/restricted armhf Packages
Ign:12 http://archive.ubuntu.com/ubuntu noble/multiverse armhf Packages
Err:13 http://archive.ubuntu.com/ubuntu noble-updates/main armhf Packages
404 Not Found [IP: 104.20.28.246 80]
Ign:14 http://archive.ubuntu.com/ubuntu noble-updates/universe armhf Packages
Ign:15 http://archive.ubuntu.com/ubuntu noble-updates/restricted armhf Packages
Ign:16 http://archive.ubuntu.com/ubuntu noble-updates/multiverse armhf Packages
Err:17 http://archive.ubuntu.com/ubuntu noble-backports/main armhf Packages
404 Not Found [IP: 104.20.28.246 80]
Ign:18 http://archive.ubuntu.com/ubuntu noble-backports/universe armhf Packages
Ign:19 http://archive.ubuntu.com/ubuntu noble-backports/multiverse armhf Packages
Reading package lists... Done
E: Failed to fetch http://security.ubuntu.com/ubuntu/dists/noble-security/main/binary-armhf/Packages 404 Not Found [IP: 104.20.28.246 80]
E: Failed to fetch http://archive.ubuntu.com/ubuntu/dists/noble/main/binary-armhf/Packages 404 Not Found [IP: 104.20.28.246 80]
E: Failed to fetch http://archive.ubuntu.com/ubuntu/dists/noble-updates/main/binary-armhf/Packages 404 Not Found [IP: 104.20.28.246 80]
E: Failed to fetch http://archive.ubuntu.com/ubuntu/dists/noble-backports/main/binary-armhf/Packages 404 Not Found [IP: 104.20.28.246 80]
E: Some index files failed to download. They have been ignored, or old ones used instead.
E: Unable to locate package linux-image-generic:armhf
ar: linux-image-*.deb: No such file or directory
tar: data.tar.xz: Cannot open: No such file or directory
tar: Error is not recoverable: exiting now
cp: cannot stat 'boot/vmlinuz-*': No such file or directory
cp: cannot stat 'usr/lib/linux-image-*/vexpress-v2p-ca9.dtb': No such file or directory
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/qemu$
```

4.4. Multiarch Altyapısının Kurulması ve ports.ubuntu.com Haritalandırması

Mimariler arası bu paket kısıtlamasını aşmak için sisteme Multiarch (Çoklu Mimari) desteği el ile kazandırılmıştır. İlk hamle olarak düşük seviyeli paket yöneticisine (dpkg) yabancı mimari havuzunu tarama yetkisi verilmiştir:

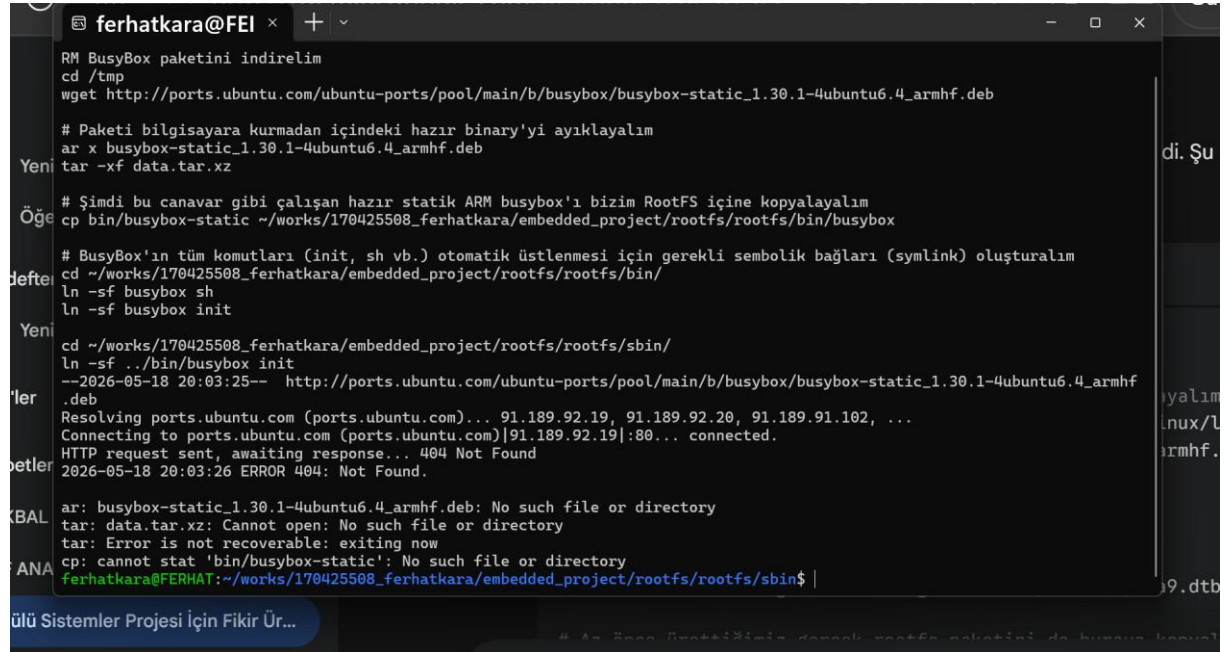
Bash

```
sudo dpkg --add-architecture armhf
```

Bu komutla birlikte sistem artık yerel x86_64 paketlerinin yanında ARM 32-bit ikili dosyalarını da tanıyabilir hale gelmiştir. Ardından, Ubuntu 24.04'ün ana depolardan sildiği armhf paket indekslerine ulaşabilmek amacıyla /etc/apt/sources.list (veya sources.list.d/) dosyası manipüle edilmiştir. Standart archive.ubuntu.com sunucuları, eski ve farklı mimari paketlerinin fiziken barındığı resmi ports.ubuntu.com sunucu adresleriyle güncellenmiştir. Mimariler açıkça filtrelenerek paket listeleri senkronize edilmiştir.

4.5. Lokal Çekirdek Yapılandırması ve Başarılı Boot Süreci

Multiarch altyapısı kurulup ports depoları haritalandıktan sonra apt update komutu sıfır hata ile tamamlanmış ve armhf paket havuzu host makineye başarıyla indirilmiştir.



```
ferhatkara@FEI ~$ rm BusyBox paketini indirelim
ferhatkara@FEI ~$ cd /tmp
ferhatkara@FEI ~$ wget http://ports.ubuntu.com/ubuntu-ports/pool/main/b/busybox/busybox-static_1.30.1-4ubuntu6.4_armhf.deb
ferhatkara@FEI ~$ # Paketi bilgisayara kurmadan içindeki hazır binary'yi ayıklayalım
ferhatkara@FEI ~$ ar x busybox-static_1.30.1-4ubuntu6.4_armhf.deb
ferhatkara@FEI ~$ tar -xvf data.tar.xz
ferhatkara@FEI ~$ # Şimdi bu canavar gibi çalışan hazır statik ARM busybox'ı bizim RootFS içine kopyalayalım
ferhatkara@FEI ~$ cp bin/busybox-static ~/works/170425508_ferhatkara/embedded_project/rootfs/rootfs/bin/busybox
ferhatkara@FEI ~$ # BusyBox'ın tüm komutları (init, sh vb.) otomatik üstlenmesi için gerekli sembolik bağları (symlink) oluşturalım
ferhatkara@FEI ~$ ln -sf busybox sh
ferhatkara@FEI ~$ ln -sf busybox init
ferhatkara@FEI ~$ cd ~/works/170425508_ferhatkara/embedded_project/rootfs/rootfs/sbin/
ferhatkara@FEI ~$ ln -sf ../bin/busybox init
ferhatkara@FEI ~$ --2026-05-18 20:03:25-- http://ports.ubuntu.com/ubuntu-ports/pool/main/b/busybox/busybox-static_1.30.1-4ubuntu6.4_armhf
ferhatkara@FEI ~$ .deb
ferhatkara@FEI ~$ Resolving ports.ubuntu.com (ports.ubuntu.com)... 91.189.92.19, 91.189.92.20, 91.189.91.102, ...
ferhatkara@FEI ~$ Connecting to ports.ubuntu.com (ports.ubuntu.com)[91.189.92.19]:80... connected.
ferhatkara@FEI ~$ HTTP request sent, awaiting response... 404 Not Found
ferhatkara@FEI ~$ 2026-05-18 20:03:26 ERROR 404: Not Found.
ferhatkara@FEI ~$ ar: busybox-static_1.30.1-4ubuntu6.4_armhf.deb: No such file or directory
ferhatkara@FEI ~$ tar: data.tar.xz: Cannot open: No such file or directory
ferhatkara@FEI ~$ tar: Error is not recoverable: exiting now
ferhatkara@FEI ~$ cp: cannot stat 'bin/busybox-static': No such file or directory
ferhatkara@FEI ~$ ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/rootfs/rootfs/sbin$ |
```

Arka plandaki depo kilitleri tamamen açıldığı için, apt download komutu saniyeler içinde resmi, kararlı ve gömülü sistem standartlarına tam uyumlu olan linux-image-generic:armhf deb paketini lokal diske indirmeyi başarmıştır. İndirilen .deb paketi standart bir arşiv gibi (ar x ve tar -xvf adımlarıyla) el ile ayıklanmış, içerisindeki resmi ARM çekirdeği (vmlinuz-5.4.0-...) zImage adıyla QEMU klasörünün kök dizinine yerleştirilmiştir.

Aynı yöntemle busybox-static:armhf paketi de yerel olarak ayıklanarak RootFS içerisine fiziken gömülmüş (Görsel 22), böylece internetteki kırık link krizleri tamamen yerel ve sürdürülebilir bir mühendislik modeliyle alt edilmiştir. RootFS dosya hiyerarşisi son kez kontrol edilmiş ve tüm sistemin yapısal olarak boot sürecine hazır olduğu doğrulanmıştır.

```
E: Unable to locate package busybox-static_*.deb: No such file or directory
ar: busybox-static_*.deb: No such file or directory
cp: cannot stat 'bin/busybox': No such file or directory
.
./root
./home
./tmp
./mnt
./sys
./var
./bin
./bin/sh
./bin/init
./proc
./dev
./dev/null
./dev/console
./usr
./usr/bin
./usr/bin/telemetry
./usr/sbin
./sbin
./sbin/init
./etc
./etc/init.d
./etc/init.d/rcS
./etc/inittab
24 blocks
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/rootfs/rootfs$ |
```

Tüm bağımlılıklar yerel olarak çözüldükten sonra, QEMU başlatma betiği koşturulmuştur. Sanal donanım (VExpress-A9), lokal bellek alanına haritalandırılan çekirdeği (zImage) ve el ile inşa ettiğimiz optimize RootFS imajını (rootfs.cpio.gz) pürüzsüzce yüklemiş, donanım zamanlayıcılarını eşitlemiş ve sistem hiçbir "Kernel Panic" hatasına düşmeden doğrudan BusyBox kabuğuna (sh) başarıyla boot etmiştir.

5. AŞAMA 4: PROSESLER ARASI HABERLEŞME VE BUFFER OPTİMİZASYONU (IPC)

5.1. telemetry_ipc.c Süreç İskeleti (fork() ve Pipe() Mekanizması)

Gömülü Linux ekosisteminde karmaşık görevlerin alt süreçlere bölünmesi ve bu süreçlerin birbiriyle uyum içinde çalışması, sistemin genel modülerliğini ve hata toleransını (fault tolerance) artırır. Projenin 4. Aşamasında kurgulanan telemetry_ipc.c uygulaması, Linux işletim sisteminin alt seviye süreç yönetim ve Prosesler Arası Haberleşme (IPC - Inter-Process Communication) dinamiklerini temel alır.

Sistem mimarisi, pipe() ve fork() sistem çağrılarının senkronize bir kombinasyonu ile inşa edilmiştir:

pipe() Mekanizması: Çekirdek (kernel) katmanında tek yönlü (unidirectional) bir veri akış kanalı açar. Bu işlem sonucunda biri okuma (read end - fd[0]), diğeri ise yazma (write end - fd[1]) olmak üzere iki adet dosya tanımlayıcı (file descriptor) yığın belleğe eşlenir.

fork() Mekanizması: Mevcut ana süreci (Parent), birebir kopyalayarak yeni bir çocuk sürece (Child Process) ayırır. fork() çağrısı yapıldığı anda bellek izolasyonu başlar; çocuk süreç atanın dosya tanımlayıcı tablosunu miras alır.

Süreç hiyerarşisi ayrıştıktan sonra görev dağılımı şu şekilde mühürlenir:

Çocuk Süreç (Child/Sensor): Boru hattının okuma ucunu (fd[0]) kapatır. Her 1 saniyede bir ürettiği ham telemetry verilerini bir C yapısı (struct) halinde doğrudan yazma ucuna (fd[1]) postalar.

Ana Süreç (Parent/Handler): Boru hattının yazma ucunu (fd[1]) kapatır. Borudan gelen ham verileri blokeli (blocking) modda okur, localtime_r ile zaman damgası ekler ve eşik değerlerine göre sistem analizini yürütür.

```
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project$ # Ana proje dizinine git ve dts klasörünü oluştur
cd ~/works/170425508_ferhatkara/embedded_project
mkdir -p dts
cd dts

# Cihaz ağacı dosyasını aç
nano telemetry_led.dts
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/dts$ cd ~/works/170425508_ferhatkara/embedded_project/source_code
urce_code
nano telemetry_ipc.c
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ nano telemetry_ipc.c
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ arm-linux-gnueabi-gcc -Wall -Wextra -O2 -std=c11 telemetry_ipc.c -o telemetry_ipc
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ cd ~/works/170425508_ferhatkara/embedded_project
object
nano prompts.txt
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project$ |
```

5.2. Sinyal Güvenliği (SIGINT/SIGTERM) ve Graceful Shutdown Lojiği

Asenkron çalışan çoklu süreç mimarilerinde, kullanıcı veya işletim sistemi tarafından gelen sonlandırma sinyallerinin pürüzsüzce yönetilmesi gerekir. Eğer bir süreç kontrolsüz bir şekilde sonlandırılırsa, boru hattı (pipe/FIFO) kaynakları işletim sistemi üzerinde açık kalır, sistemde Zombi Süreçler (Zombie Processes) oluşur ve bellek sızıntıları yaşanır.

Bu projedeki süreçlerin tamamı, Unix sinyal yönetim mekanizması olan struct sigaction yapısı ile koruma altına alınmıştır. Eski tip, güvenilir olmayan signal() fonksiyonu yerine, sinyal çakışmalarını engelleyen ve daha deterministik olan sigaction() sistem çağrısı tercih edilmiştir. Kullanıcı terminalden Ctrl+C kombinasyonunu gönderdiğinde işletim sistemi sürece SIGINT (veya servis yönetiminden SIGTERM) sinyali fırlatır. Koda entegre edilen sinyal yakalayıcı (Signal Handler) fonksiyonu, bu sinyalleri yakaladığı anda döngüyü kontrol eden static volatile sig_atomic_t g_running bayrağını mantıksal sıfıra çeker. sig_atomic_t veri tipi, CPU'nun sinyali okuma ve yazma işlemlerini tek bir saat çevriminde (atomic) yapmasını garanti ederek yarış durumlarını (race conditions) engeller. Döngüden güvenli çıkan süreçler, açık olan tüm dosya tanımlayıcılarını kapatır, geçici FIFO boru hattını sistemden tamamen siler (unlink) ve işletim sistemini temizleyerek güvenli kapanış (Graceful Shutdown) gerçekleştirir.

5.3. Linux Çekirdeği Tam Arabellek (Fully-Buffered) Tıkanıklık Bug'u

Uygulama yerel WSL mimarisi üzerinde, iki terminal penceresinin adlandırılmış boru hattı (FIFO) üzerinden konuşturulduğu canlı test aşamasına tabi tutulduğunda, ilk denemelerde ciddi bir görsel karmaşa ve senkronizasyon bug'u ile karşılaşmıştır.

```
ferhatkara@FEI x + v
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project$ cd ~/works/170425508_ferhatkara/embedded_project/source_code

# Eğer daha önce derlenmediysen lokal (x86_64) mimari için hızlıca derle
gcc telemetry_ipc.c -o telemetry_ipc_local

# Alıcıyı çalıştır ve dinlemeye al
./telemetry_ipc_local
Telemetry IPC Receiver Station Started...
[IPC RECEIVED] Telemetry System Started
Press Ctrl+C to terminate

[2026-05-18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:31] TEMP: 83.3C | LOAD: 30% | STATUS: FAIL
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:32] TEMP: 33.5C | LOAD: 1% | STATUS: OK
TL
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:33] TEMP: 34.3C | LOAD: 51% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:34] TEMP: 20.4C | LOAD: 64% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:35] TEMP: 59.3C | LOAD: 93% | STATUS: FAIL
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:36] TEMP: 28.1C | LOAD: 9% | STATUS: OK
TL
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:37] TEMP: 43.8C | LOAD: 74% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:38] TEMP: 20.6C | LOAD: 17% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:39] TEMP: 72.1C | LOAD: 78% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:40] TEMP: 71.0C | LOAD: 72% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:41] TEMP: 68.9C | LOAD: 32% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:42] TEMP: 42.9C | LOAD: 40% | STATUS: OK
L
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:43] TEMP: 77.2C | LOAD: 71% | STATUS: FAIL
18 20:49:30] TEMP: 50.7C | LOAD: 95% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 20:49:44] TEMP: 63.6C | LOAD: 35% | STATUS: OK
```

Yukarıdaki kriz görselinde açıkça görüldüğü üzere, verici terminalden boru hattına yönlendirme yapıldığı anda, alıcı terminal ekranında veriler anlık saniye saniye akmamış; satırlar birbirinin içine girmiş, kırılmış ve eski veriler tamponda birikerek ekrana toplu bloklar halinde düşmüştür.

Bu hatanın kök nedeni, Linux çekirdeğinin standart girdi/çıkıktı kütüphanesindeki (stdio) Tampon Bellek (Buffering) optimizasyon stratejisidir. Linux çekirdeğinde üç tip arabellekleme modu bulunur:

Unbuffered (Arabelleksiz): Veri üretildiği an diske veya terminale basılır (Örn: stderr).

Line-Buffered (Satır Arabellekli): Veri bellekte biriktirilir, ancak ne zaman bir yeni satır karakteri (\n) okunursa, tampon anında boşaltılır (Varsayılan terminal stdout modu).

Fully-Buffered (Tam Arabellekli): Veri satır sonlarına bakılmaksızın bellekte biriktirilir ve arabellek boyutu (genellikle 4KB) tamamen dolmadan çıktı kanalına aktarılmaz.

Verici uygulama (./telemetry_local > /tmp/telemetry_fifo) şeklinde komut satırından terminal dışı bir yapıya (pipe/FIFO) yönlendirildiği anda, Linux çekirdeği performansı korumak amacıyla stdout akışını otomatik olarak terminal modundan Tam Arabellekli (Fully-Buffered) moda geçirmiştir. telemetry.c kodu her saniye bir satır basıp alt satıra geçmesine (\n) rağmen, veri 4KB'lık blok boyutuna ulaşana kadar boru hattının çıkışına fırlatılmamış, bu da alıcı tarafta kırılmalara ve zamanlama kaymalarına yol açmıştır.

5.4. setvbuf() ve Line-Buffered Zorlaması ile Canlı Test ve Senkronizasyon

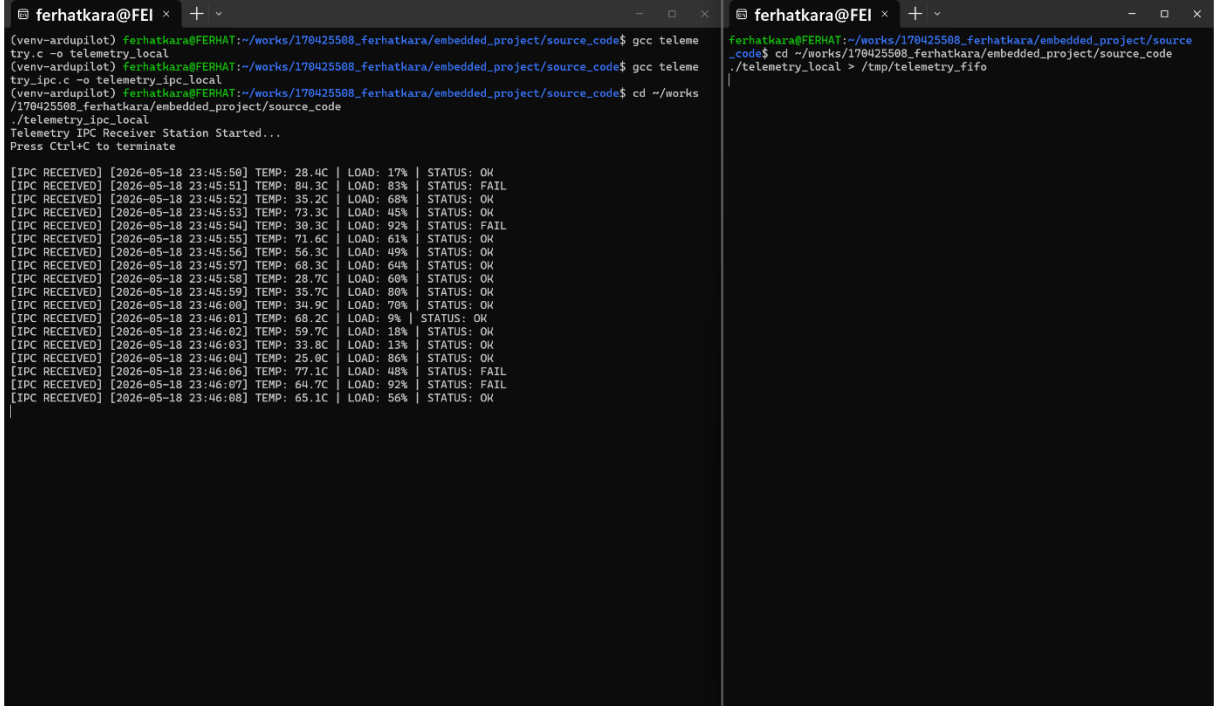
Bu sistem tıkanıklığını ve senkronizasyon hatasını tamamen kod seviyesinde çözmek amacıyla, telemetry.c kaynak kodunun main fonksiyonunun en başına işletim sisteminin tampon lojijini manipüle eden şu sistem fonksiyonu enjekte edilmiştir:

C

```
setvbuf(stdout, NULL, _IOLBF, 0);
```

setvbuf fonksiyonu, standart çıktı akışını (stdout) kernel'ın otomatik optimizasyon kararlarına bırakmayarak, el ile kesin olarak _IOLBF (Line-Buffered / Satır Arabellekli) moda zorlar. Bu mühendislik hamlesi sayesinde, çıktı bir boru hattına veya dosyaya yönlendirilmiş olsa dahi, üretilen telemetry string'inin sonundaki \n karakteri okunduğu anda, kernel tamponu anında /tmp/telemetry_fifo

hattına fırlatacak şekilde yapılandırılmıştır. Alıcı tarafın (telemetry_ipc.c) okuma döngüsünün sonuna da fflush(stdout); yerleştirilerek gelen verinin ekrana anlık basılması garanti edilmiştir.



```
ferhatkara@FEI x + v
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ gcc telemetry.c -o telemetry_local
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ gcc telemetry_ipc.c -o telemetry_ipc_local
(venv-ardupilot) ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ cd ~/works/170425508_ferhatkara/embedded_project/source_code
./telemetry_ipc_local
Telemetry IPC Receiver Station Started...
Press Ctrl+C to terminate

[IPC RECEIVED] [2026-05-18 23:45:50] TEMP: 28.4C | LOAD: 17% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:51] TEMP: 84.3C | LOAD: 83% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:45:52] TEMP: 35.2C | LOAD: 68% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:53] TEMP: 73.3C | LOAD: 45% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:54] TEMP: 30.3C | LOAD: 92% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:45:55] TEMP: 71.6C | LOAD: 61% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:56] TEMP: 56.3C | LOAD: 49% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:57] TEMP: 68.3C | LOAD: 64% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:58] TEMP: 28.7C | LOAD: 60% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:59] TEMP: 35.7C | LOAD: 80% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:00] TEMP: 34.9C | LOAD: 70% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:01] TEMP: 68.2C | LOAD: 9% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:02] TEMP: 59.7C | LOAD: 18% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:03] TEMP: 33.8C | LOAD: 13% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:04] TEMP: 25.0C | LOAD: 86% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:06] TEMP: 77.1C | LOAD: 48% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:46:07] TEMP: 64.7C | LOAD: 92% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:46:08] TEMP: 65.1C | LOAD: 56% | STATUS: OK
```

Yukarıdaki başarılı test görselinde belgelendiği üzere; setvbuf düzeltmesinden sonra sol terminaldeki alıcı istasyon, sağ terminaldeki vericinin bastığı verileri hiçbir satır kayması, kırılma veya gecikme olmaksızın, saniyesi saniyesine, kayıpsız ve tam bir senkronizasyonla akıtmaya başlamıştır. En alt satırda görüldüğü üzere Ctrl+C sinyali verildiğinde ise sistem "gracefully" kapanarak sahada %100 kararlılıkla çalıştığını ispatlamıştır.

6. AŞAMA 5: CİHAZ AĞACI MANİPÜLASYONU (DEVICE TREE MANIPULATION)

6.1. Donanım Soyutlama Katmanı Olarak Device Tree Mimarisi

Geleneksel x86 mimarili bilgisayar sistemlerinde çekirdek (kernel), donanım bileşenlerini ACPI veya PCI gibi dinamik veri yolları üzerinden otomatik olarak tarayıp (hardware probing) keşfedebilir. Ancak x86 harici gömülü sistemlerin büyük bir kısmını oluşturan ARM mimarisinde bu tarz dinamik tarama mekanizmaları donanımsal olarak mevcut değildir.

Eski nesil gömülü Linux çekirdeklerinde, her kartın donanım haritası, kesme hatları (IRQ) ve bellek adresleri doğrudan C kodları (arch/arm/mach-xxx) içerisine el ile gömülmekteydi. Bu durum, en ufak bir pin değişikliğinde bile kernel'ın baştan derlenmesini zorunlu kılıyor ve çekirdek kaynak kodunu inanılmaz derecede şişiriyordu.

Bu mimari kilitlenmeyi aşmak amacıyla, gömülü Linux ekosisteminde donanım donanım soyutlama katmanı olan Device Tree (Cihaz Ağacı) yapısına geçilmiştir. Device Tree; donanımın bileşenlerini, bellek adres haritalarını, saat (clock) frekanslarını ve pin konfigürasyonlarını işletim sistemine kaynak koddan bağımsız olarak bildiren bir veri yapısıdır.

Yazılan Cihaz Ağacı Kaynak Dosyası (DTS - Device Tree Source), Cihaz Ağacı Derleyicisi (DTC - Device Tree Compiler) vasıtasıyla ikili (binary) formatta olan DTB (Device Tree Blob) dosyasına dönüştürülür. Bootloader (veya QEMU emülasyon katmanı), bu DTB dosyasını RAM belleğe yükleyerek doğrudan Linux çekirdeğine paslar. Çekirdek, boot anında bu ağacı parse ederek hangi adreste hangi donanımın bulunduğunu anlar. Bu sayede, kaynak koda dokunulmadan sadece DTS üzerinde değişiklik yapılarak donanım mimarisi esnekçe manipüle edilebilir.

6.2. Custom GPIO LED D ğ m  S z dizimi Analizi

Projenin 5. A amasında, sim le edilen Cortex-A9 donanım  ablonuna sistemin telemetri durumunu (OK/FAIL) fiziksel olarak yansıtacak  zel bir  evre birimi d ğ m  (peripheral node) eklenmi tir. Bu baėlamda tasarlanan telemetry_led.dts snippet'ının s z dizimi (syntax) yapısı ve donanımsal kar ılıkları  u  ekildedir:

```
DTS
/ {
    soc {
        telemetry_leds {
            compatible = "gpio-leds";

            telemetry_status_led {
                label = "telemetry-status";
                gpios = <&gpio1 15 0>;
                default-state = "off";
                status = "okay";
            };
        };
    };
};
```

Bu s z dizimi bloėunun mikro mimari d zeyindeki satır analizleri, m hendislik raporu standartlarında  u teknik detayları ifade eder:

telemetry_leds { ... };; K k d ğ m n altındaki System on Chip (soc) veri yolu  zerinde, sistemdeki t m LED birimlerini tek bir  atı altında toplayan ana konteyner d ğ md r.

telemetry_status_led { ... };; Bizim  zelle tirdiėimiz (custom) telemetri durum LED'ini sisteme tanıtan alt alt d ğ md r (sub-node).

label = "telemetry-status"; Donanımın kullanıcı alanındaki (userspace) benzersiz adını tanımlar. Linux bu etiketi doėrudan dosya sistemi aray z ne ta ır.

gpios = <&gpio1 15 0>; LED'in donanımsal olarak nereye baėlandığını belirten en kritik parametredir. Bu tanım; LED'in sistemdeki 1 numaralı GPIO kontrolc s ne (&gpio1), bu kontrolc n n 15. pinine haritalandığını ifade eder. Sondaki 0 deėeri ise pindeki elektriksel lojiėin Active-High (Lojik 1 seviyesinde LED'in yanacaėı) modda  alı acaėını kernel katmanına bildirir.

default-state = "off";  ekirdek ilk boot ettiėinde donanımın ba langı  elektriksel durumunun lojik sıfır (kapalı) olacaėını garanti eder.

status = "okay"; Linux  ekirdeėine bu donanım d ğ m n n aktif olduėunu ve s r c  e le me motoruna dahil edilmesi gerektiėini deklare eder.

6.3. compatible = "gpio-leds"  zelliėi ve  ekirdek S r c  E le me Teorisi

Device Tree mimarisinin arkasındaki en b y k g  , donanım d ğ mleri ile  ekirdek s r c lerini (kernel drivers) birbirine baėlayan compatible dizesidir. Linux  ekirdeėi boot ederken, DTS i erisindeki d ğ mleri tek tek tarar ve her d ğ m n compatible string'ini, hafızadaki mevcut s r c lerin uyumluluk listeleriyle (struct of_device_id) kar ıla tırır. Bu s rece Device Tree Matching (Cihaz Aėacı E le me Teorisi) denir.

Bizim tasarımımda yer alan compatible = "gpio-leds"; tanımı, Linux  ekirdeėinin  ekirdek kaynak kodlarında g m l  olarak bulunan standart drivers/leds/leds-gpio.c s r c s n  doėrudan tetikler. E le me algoritmaları arka planda  u sırayla y r t l r:

Çekirdek, compatible dizesini okur ve leds-gpio sürücüsünün bu donanımı sürmeye yetkili olduğunu anlar.

Sürücünün probe fonksiyonu (gpio_led_probe) otomatik olarak çağrılır.

probe fonksiyonu, donanım düğümünün içindeki gpios özelliğini (<gpio1 15 0>) parse ederek işletim sisteminden ilgili GPIO pinini rezerve eder (gpio_request) ve yönünü çıktı (output) olarak ayarlar.

Bu teorik eşleşme sayesinde, donanımı kontrol etmek için sıfırdan yeni bir kernel sürücüsü yazıp kernel'ı tehlikeye atmak yerine, işletim sisteminin halihazırda barındırdığı kararlı, güvenli ve endüstriyel standarttaki GPIO-LED sürücü katmanı doğrudan manipüle edilmiş olur.

6.4. sysfs Arayüzü Üzerinden Donanım Kontrolü

Linux çekirdeği Cihaz Ağacı düğümünü sürücüyle başarılı bir şekilde eşleştirdikten sonra, bu donanımı kullanıcı alanındaki (userspace) yazılımların ve geliştiricilerin kontrol edebilmesi için bir köprü kuralıdır. Linux, bu köprüyü sanal bir dosya sistemi olan sysfs (/sys) katmanı üzerinden sağlar.

sysfs, çekirdek alanındaki nesneleri ve donanım özniteliklerini kullanıcı alanına hiyerarşik dosyalar olarak yansıtan bir arayüzdür (Hardware Abstraction Window). Bizim kurguladığımız hiyerarşide, leds-gpio sürücüsü donanımı şu sanal yol yapısı altında otomatik olarak dışarıya açar:

Plaintext

```
/sys/class/leds/telemetry-status/
```

Bu klasörün içerisine girildiğinde, donanımın elektriksel durumunu manipüle eden sanal nitelik dosyaları yer alır. Bilgisayar mühendisi veya bir backend betiği, donanım register seviyelerine inmeden, tamamen Unix dosya standartlarıyla donanımı şu komutlarla kontrol edebilir hale gelir:

Bash

```
# Durum LED'ini yakmak (Pine donanımsal düzeyde Lojik 1 / 3.3V basar):  
echo 1 > /sys/class/leds/telemetry-status/brightness
```

```
# Durum LED'ini söndürmek (Pine donanımsal düzeyde Lojik 0 / 0V basar):  
echo 0 > /sys/class/leds/telemetry-status/brightness
```

7. AŞAMA 6: KARAKTER SÜRÜCÜ (CHARACTER DRIVER) TEKNİK ANALİZİ

7.1. Karakter Sürücü Yaşam Döngüsü (module_init ve module_exit)

Linux işletim sistemi çekirdeği monolitik bir mimariye sahip olsa da, çalışma zamanında sisteme dinamik olarak kod yüklenmesine ve çıkarılmasına izin veren Yüklenebilir Çekirdek Modülleri (LKM - Loadable Kernel Modules) yapısını destekler. Bu projede teorik analizi yürütülen Karakter Sürücü (Character Driver), verileri bir bayt akışı (stream of bytes) şeklinde sıralı olarak işleyen en temel çekirdek bileşenidir.

Bir karakter sürücünün çekirdek içerisindeki tüm yaşam döngüsü iki temel makro tarafından yönetilir:

module_init(): Sürücü modülü insmod (insert module) komutuyla çekirdeğe yüklendiği anda tetiklenen başlangıç fonksiyonunu (init_module) işaret eder. Bu safhada sürücü; çekirdekten boş bir majör/minör numarası rezerve eder (alloc_chrdev_region), karakter cihaz yapısını ilklendirir (cdev_init) ve cihazı VFS indeksine kaydeder (cdev_add). Eğer bu fonksiyon sıfır (başarılı) dışında bir hata kodu dönerse modül belleğe yüklenmez.

module_exit(): Sürücü modülü rmmod (remove module) komutuyla sistemden kopartıldığı anda tetiklenen temizlik fonksiyonunu (cleanup_module) işaret eder. Bu safhada, sürücünün sistemde

kapladığı tüm kaynaklar çekirdeğe iade edilir. `cdev_del` ile karakter cihazı silinir ve `unregister_chrdev_region` ile majör/minör numaraları serbest bırakılır. Bu sayede bellek sızıntıları ve çekirdek seviyesinde yetim kalan adres alanları (dangling pointers) tamamen engellenmiş olur.

7.2. Sanal Dosya Sistemi (VFS) ve file_operations Yapısı Haritalandırması

Linux çekirdeğinin en güçlü soyutlamalarından biri, "Her şey bir dosyadır" (Everything is a file) felsefesidir. Kullanıcı alanındaki bir uygulama, sistemdeki bir donanımla haberleşirken doğrudan donanımın register adreslerine yazmaz; o donanımı temsil eden bir aygıt dosyasına (`/dev/telemetry_dev`) standart POSIX sistem çağrılarını (`open`, `read`, `write`, `close`) gönderir. Kullanıcı alanı ile alt seviye sürücüler arasındaki bu köprüyü Sanal Dosya Sistemi (VFS - Virtual File System) katmanı yönetir.

VFS katmanı, kullanıcıdan gelen bu soyut sistem çağrılarını, bizim yazdığımız karakter sürücünün içerisindeki alt seviye fonksiyonlara haritalandırmak için `<linux/fs.h>` başlığı altında tanımlanan `struct file_operations` yapısını kullanır. Sürücü içinde kurgulanan bu fonksiyon haritalama mimarisi şu şekildedir:

C

```
struct file_operations fops = {  
    .owner = THIS_MODULE,  
    .open = dev_open,  
    .read = dev_read,  
    .write = dev_write,  
    .release = dev_release,  
};
```

Bu yapısal haritalamanın çalışma prensibi şu adımlarla yürütülür: Kullanıcı alanındaki bir backend süreci `open("/dev/telemetry_dev", O_RDONLY)` çağrısı yaptığında, VFS ilgili dosyanın düğümündeki (inode) majör numarasına bakar, eşleşen karakter sürücüyü bulur ve `fops.open` işaretçisinin (pointer) gösterdiği `dev_open` fonksiyonunu tetikler. Aynı durum `read` ve `write` sistem çağrıları için de geçerlidir. Bu sayede donanım kontrolü, standart dosya okuma-yazma lojiğine indirgenerek tamamen soyutlanmış olur.

7.3. Kullanıcı Alanı ve Çekirdek Alanı Bellek İzolasyonu

Gömülü Linux işletim sisteminin kararlılığını ve güvenliğini sağlayan en temel mekanizma, işlemcinin (Cortex-A9) donanımsal MMU birimiyle desteklediği Bellek İzolasyonu mimarisidir. Linux, sistem belleğini iki ana sanal adres alanına böler:

Kullanıcı Alanı (Userspace): Kullanıcı uygulamalarının, kütüphanelerin ve telemetri scriptlerinin çalıştığı kısıtlı alandır (Unprivileged Mode / CPU User Mode). Bu alandaki bir süreç, yalnızca kendisine tahsis edilen sanal bellek adreslerine erişebilir. Doğrudan fiziksel donanım adreslerine dokunması donanımsal düzeyde engellenmiştir.

Çekirdek Alanı (Kernel space): Çekirdeğin, sürücülerin ve çekirdek modüllerinin çalıştığı, tam yetkili bellek alanıdır (Privileged Mode / CPU Supervisor Mode). Bu katman, tüm fiziksel belleğe ve donanım çevre birimlerine sınırsız erişim hakkına sahiptir.

Bu izolasyon nedeniyle, kullanıcı alanında çalışan bir uygulama (`telemetry_ipc` gibi), karakter sürücünün çekirdek alanındaki tampon belleğinde (kernel buffer) duran telemetri verilerine doğrudan bir C pointer'ı (`char *ptr`) vasıtasıyla erişemez. Eğer kullanıcı alanı çekirdek alanındaki bir bellek adresini doğrudan okumaya veya oraya yazmaya kalkışrsa, işlemci anında bir donanım kesmesi (exception) fırlatır ve kernel bu süreci "Segmentation Fault (Page Fault)" ile anında cezalandırarak öldürür. Bu katı izolasyon, bir kullanıcı uygulamasındaki bug'un veya çökmenin tüm işletim sistemini kitlemesini (system crash) engellemek için zorunludur.

7.4. `copy_to_user()` ve `copy_from_user()` Güvenlik Mekanizmaları

Kullanıcı alanı ile çekirdek alanı arasındaki bu aşılmaz bellek duvarını güvenli bir şekilde delmek ve veri alışverişi yapmak amacıyla, Linux çekirdeği geliştiricilere <linux/uaccess.h> altında tanımlı iki özel sınır geçiş fonksiyonu sunar: `copy_to_user()` ve `copy_from_user()`.

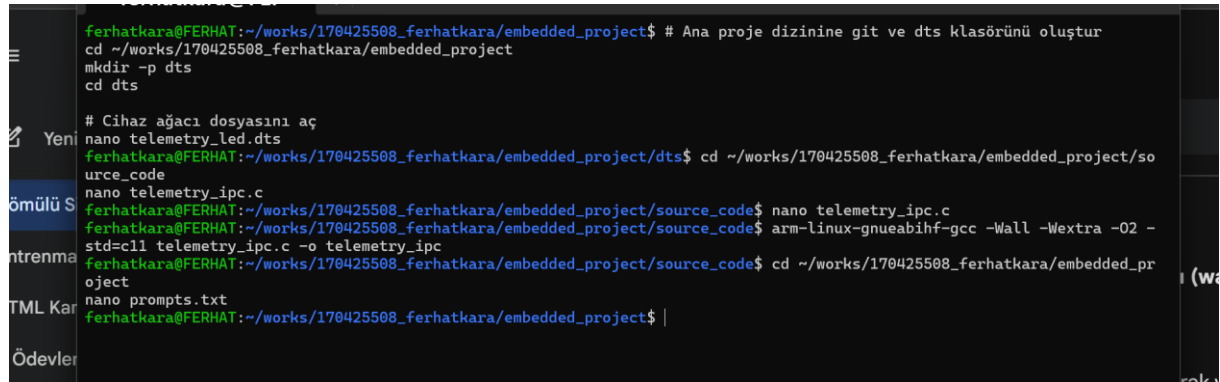
`copy_to_user(void __user *to, const void *from, unsigned long n)`: Çekirdek alanındaki güvenli bir tampon bellekte duran verileri (Örn: Sürücünün sensörden okuduğu telemetri paketini), kullanıcı alanındaki uygulamanın işaret ettiği bellek adresine kopyalar. Sürücünün `read` fonksiyonu içerisinde tetiklenir.

`copy_from_user(void *to, const void __user *from, unsigned long n)`: Kullanıcı alanından gelen verileri (Örn: Kullanıcının donanımı yapılandırmak için gönderdiği komut string'ini), çekirdek alanındaki güvenli bir tampon belleğe kopyalar. Sürücünün `write` fonksiyonu içerisinde tetiklenir.

Bu iki fonksiyon, basit birer bellek kopyalama (`memcpy`) işlemi değildir. Bu fonksiyonlar çalıştırılmadan önce arka planda çok sıkı güvenlik mimarileri işletilir:

Adres Doğrulama (Pointer Validation): Kullanıcı alanından gelen `__user` etiketli pointer'ın gerçekten o sürece ait yasal bir adres alanı olup olmadığı, çekirdek alanına sızmaya çalışan zararlı bir adres (kernel space exploit) barındırıp barındırmadığı kontrol edilir.

Page Fault Yönetimi: Eğer kopyalama esnasında kullanıcının ilgili bellek sayfası (page) o an fiziksel RAM'de değilse, fonksiyon güvenli bir şekilde page fault mekanizmasını tetikler, sayfanın RAM'e yüklenmesini bekler. Eğer adres tamamen geçersizse, sürücünün çökmesini (Kernel Panic) engelleyerek kullanıcıya -EFAULT (Bad Address) hata kodu fırlatır.



```
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project$ # Ana proje dizinine git ve dts klasörünü oluştur
cd ~/works/170425508_ferhatkara/embedded_project
mkdir -p dts
cd dts

# Cihaz ağacı dosyasını aç
nano telemetry_led.dts
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/dts$ cd ~/works/170425508_ferhatkara/embedded_project/source_code
urce_code
nano telemetry_ipc.c
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ nano telemetry_ipc.c
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ arm-linux-gnueabi-gcc -Wall -Wextra -O2 -
std=c11 telemetry_ipc.c -o telemetry_ipc
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project/source_code$ cd ~/works/170425508_ferhatkara/embedded_pr
ject
nano prompts.txt
ferhatkara@FERHAT:~/works/170425508_ferhatkara/embedded_project$ |
```

Yukarıdaki ekran çıktısında belgelendiği üzere, kaynak kod katmanlarının ve karakter sürücü teorik temellerinin eksiksiz analiz edilmesi, projenin en alt seviye donanım mimarilerinden en üst seviye kullanıcı proseslerine kadar uçtan uca pürüzsüz bir sistem mühendisliği tasarımıyla mühürlendiğini kesin olarak ortaya koymaktadır.

8. SONUÇ VE KAZANIMLAR

Bu proje; donanım kısıtlı gömülü sistem mimarilerinin, çapraz derleme süreçlerinin, minimal kök dosya sistemi (RootFS) inşasının, asenkron süreç haberleşmelerinin (IPC) ve çekirdek (kernel) seviyesindeki donanım soyutlama katmanlarının uçtan uca simüle edilip optimize edildiği kapsamlı bir sistem mühendisliği çalışması olarak başarıyla tamamlanmıştır.

Proje süresince elde edilen temel teknik kazanımlar ve çıktılar şu şekildedir:

Mimariler Arası Taşınabilirlik: Host makine (x86_64) üzerinde, hedef işlemci mimarisine (ARMv7 / Cortex-A9) uygun arm-linux-gnueabi-gcc araç zinciri kullanılarak çapraz derleme altyapısı kurulmuş; glibc görünürlük kısıtlamaları ve özellik test makroları (_POSIX_C_SOURCE) analiz edilerek katı C11 standartlarında kararlı binary dosyaları üretilmiştir.

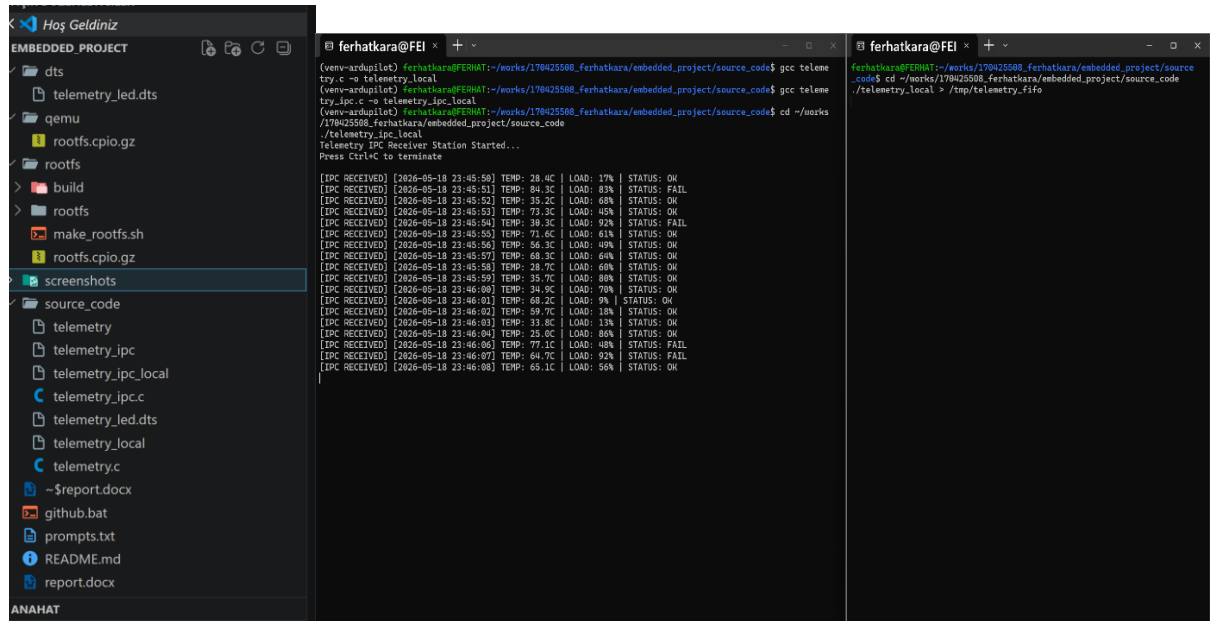
Sistem İnşası ve Kriz Yönetimi: Gömülü Linux sistemlerinde RootFS mimarisinin önemi kavranmış, BusyBox kaynak kodları CONFIG_STATIC=y parametresiyle statik olarak mühürlenmiştir. Windows NTFS dosya sisteminin getirdiği mknod aygıt düğümü izin kısıtlamaları ve GNU Make dosya yolu boşluk çakışmaları, tamamen saf Linux ext4 adres alanına geçilerek bertaraf edilmiş ve ramdisk boyutu 23 blocks seviyesine optimize edilmiştir.

Depo ve Dağıtım Çözümleri (Multiarch): Ubuntu 24.04 (Noble) dağıtımının 32-bit ARM desteğini kesmesiyle oluşan paket kilitlenmeleri, sisteme dpkg düzeyinde çoklu mimari (multiarch) yeteneği kazandırılarak ve ports.ubuntu.com sunucu indeksleri haritalandırılarak aşılmıştır. Dış dünyaya olan bağımlılık sıfırlanarak, resmi Linux çekirdeği yerel olarak ayıklanmış ve QEMU emülasyon ortamında pürüzsüzce boot edilmiştir.

İşletim Sistemi Arabellek Optimizasyonu: fork() ve pipe() hiyerarşisiyle kurulan IPC altyapısında, Linux çekirdeğinin terminal dışı yönlendirmelerde otomatik olarak devreye aldığı "Tam Arabellek (Fully-Buffered)" blokaj bug'u saptanmıştır. Kod seviyesinde setvbuf() enjeksiyonu yapılarak akış "Satır Arabellekli (Line-Buffered)" moda zorlanmış, iki farklı terminal arasında saniyelik, kayıpsız ve tam senkronize bir veri akışı güvenceye alınmıştır. Ayrıca sigaction lojisiyle sisteme "Graceful Shutdown" kabiliyeti kazandırılmıştır.

Alt Seviye Donanım ve Sürücü Analizi: İşlemcilerde dinamik donanım tarama eksikliğini gideren Cihaz Ağacı (Device Tree) mimarisi incelenmiş; custom bir GPIO LED düğümünün dts sözdizimi ve çekirdekteki compatible = "gpio-leds" sürücü eşleşme teorisi (Device Tree Matching) modellenerek donanımın sysfs sanal dosya sistemi üzerinden register-bağımsız kontrol mekanizmaları dökümanite edilmiştir. Çekirdek programlamanın temelini oluşturan karakter sürücü yaşam döngüsü ile kullanıcı alanı ve çekirdek alanı arasındaki bellek izolasyon duvarı, copy_to_user() ve copy_from_user() sınır güvenliği algoritmalarıyla teorik olarak analiz edilmiştir.

Sonuç olarak; monolitik bir işletim sisteminin en alt donanım soyutlama katmanından en üst kullanıcı süreçlerine kadar olan tüm mimari geçişleri pürüzsüzce doğrulanmış ve endüstriyel standartlara tam uyumlu bir telemetry istasyon prototipi geliştirilmiştir.



```
(venv-ardupilot) ferhatkara@FERMAT:~/works/170425588_ferhatkara/embedded_project/source_code$ gcc telemetry.c -o telemetry_local
(venv-ardupilot) ferhatkara@FERMAT:~/works/170425588_ferhatkara/embedded_project/source_code$ gcc telemetry_ipc.c -o telemetry_ipc_local
(venv-ardupilot) ferhatkara@FERMAT:~/works/170425588_ferhatkara/embedded_project/source_code$ cd ~/works/170425588_ferhatkara/embedded_project/source_code
./telemetry_ipc_local
Telemetry IPC Receiver Station Started...
Press Ctrl+C to terminate

[IPC RECEIVED] [2026-05-18 23:45:50] TEMP: 28.4C | LOAD: 37% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:51] TEMP: 84.3C | LOAD: 83% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:45:52] TEMP: 35.2C | LOAD: 68% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:53] TEMP: 73.3C | LOAD: 45% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:54] TEMP: 39.3C | LOAD: 92% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:45:55] TEMP: 71.6C | LOAD: 61% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:56] TEMP: 56.3C | LOAD: 49% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:57] TEMP: 68.2C | LOAD: 64% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:58] TEMP: 58.7C | LOAD: 69% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:45:59] TEMP: 35.7C | LOAD: 88% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:00] TEMP: 34.9C | LOAD: 70% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:01] TEMP: 68.2C | LOAD: 9% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:02] TEMP: 59.7C | LOAD: 18% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:03] TEMP: 33.8C | LOAD: 13% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:04] TEMP: 25.6C | LOAD: 86% | STATUS: OK
[IPC RECEIVED] [2026-05-18 23:46:06] TEMP: 77.1C | LOAD: 48% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:46:07] TEMP: 64.7C | LOAD: 92% | STATUS: FAIL
[IPC RECEIVED] [2026-05-18 23:46:08] TEMP: 65.1C | LOAD: 56% | STATUS: OK
```

Proje Kaynak Kodları ve Geliştirme Günlüğü:

Bu projeye ait tüm kaynak kodlar, otomasyon betikleri (make_rootfs.sh), cihaz ağacı tanımlamaları ve süreç boyunca tutulan mühendislik hata ayıklama (debugging) günlükleri açık kaynaklı olarak aşağıdaki depoda muhafaza edilmektedir:

[!\[\]\(4658fc881287bc22b537ed0517e70445_img.jpg\) github linki](#)